Ugali Preparation Language (UPL) — Lexical Analysis

This document answers parts (a)–(j) of the assignment. The implementation is in `Lexer.flex`, with Main.scala as a small demo driver and Tests.scala as the verification program for parts (d)–(h).

Build and run

The project needs JFlex, a JDK, and Scala 3 (the original `Main.scala` uses Scala 3's constructor syntax).

```
make run    # runs the original demo input
make test   # runs every example used in parts (d)-(h)
make clean
```

---

a) Lexical specification

Category	Lexemes	Token names
Reserved words	`prepare ingredients boil water add flour stir press turn cook check consistency shape serve ugali cups minutes`	`PREPARE`, `INGREDIENTS`, `BOIL`, `WATER`, `ADD`, `FLOUR`, `STIR`, `PRESS`, `TURN`, `COOK`, `CHECK`, `CONSISTENCY`, `SHAPE`, `SERVE`, `UGALI`, `CUPS`, `MINUTES`
Integer literal	`0`, `4`, `25`, …	`NUMBER`
Identifier	`temperature`, `maize_flour`, `step2`, `cooking_time`	`IDENTIFIER`
Punctuation	`;`	`SEMICOLON`
Ignored	spaces, tabs, line breaks, `// ...` comments, `/* ... */` comments	—
Errors	any other character, unterminated `/* ... */`	`ILLEGAL`, `UNTERMINATED_COMMENT`

A lexeme is one concrete piece of input text. Its token is the category returned by the scanner. For example, in `cook 8 minutes;` the lexemes are `cook`, `8`, `minutes`, `;` and the corresponding token sequence is `COOK NUMBER MINUTES SEMICOLON`.

---

b) Regular expressions

Let `letter = [a-zA-Z]`, `digit = [0-9]`, and `alnum = [a-zA-Z0-9_]`.

Lexical item	Regular expression
Integer literal	`digit+`
Identifier	`letter (letter | digit | _)*` — in JFlex: `[a-zA-Z][a-zA-Z0-9_]*`
Whitespace	`[ \t\n\r]+`
Single-line comment	`//[^\r\n]*`
Reserved words	`"prepare" | "ingredients" | "boil" | "water" | "add" | "flour" | "stir" | "press" | "turn" | "cook" | "check" | "consistency" | "shape" | "serve" | "ugali" | "cups" | "minutes"`
Semicolon	`;`
Illegal character	`.` (anything not matched by a previous rule)

The identifier rule requires a letter first, exactly as specified by `letter (letter | digit | _)*`. (The original version allowed a leading underscore; it was corrected to match the assignment's rule.)

For the extension in part (h), a multiline comment is handled with lexical states rather than one monolithic regular expression:

```
%x COMMENT

"/*"        { yybegin(COMMENT); }
<COMMENT> {
  "*/"      { yybegin(YYINITIAL); }
  [^*]+     { /* comment body */ }
  "*"       { /* lone star inside the comment */ }
  <<EOF>>   { return Token.UNTERMINATED_COMMENT; }
}
```

Equivalently, the whole comment is matched by `/\* ([^*] | \*+ [^*/])* \*+/`, but lexical states make the unterminated case easy to detect.

---

c) JFlex implementation

The complete specification is `Lexer.flex`. It satisfies the requirements as follows:

- `%class Lexer`, `%type Token`, `%unicode`, `%line`, `%column` generate the scanner and enable accurate line/column tracking.
- All reserved words are recognised before the general identifier rule.
- Integer literals return `Token.NUMBER`; their value is retained in `numberValue`.
- Identifiers return `Token.IDENTIFIER`.
- `;` returns `Token.SEMICOLON`.
- Whitespace and comments are consumed without returning a token.
- `line()` and `column()` return one-based positions.
- Anything unmatched returns `Token.ILLEGAL`; the driver prints its character, line and column.
- `%x COMMENT` plus `<<EOF>>` detects an unterminated multiline comment.

---

d) Token attributes

The scanner keeps both the token type and the integer value:

```
private int numberValue;
private String numberText;

public int numberValue()   { return numberValue; }
public String numberText() { return numberText; }

[0-9]+ {
  numberText = yytext();
  try {
    numberValue = Integer.parseInt(numberText);
  } catch (NumberFormatException e) {
    // The token is still returned if the literal is outside int range.
    numberValue = 0;
  }
  return Token.NUMBER;
}
```

For `boil 4 cups water;` the scanner therefore produces:

```
BOIL
NUMBER(4)
CUPS
WATER
SEMICOLON
```

Storing `4` is important because later phases must use the number itself, not just the fact that a number occurred. The parser needs it to build an expression/AST node, semantic analysis needs it for range or type checks, and code generation needs it as an operand. Re-reading the source text during every later phase would be slower and would duplicate lexical work. In a parser-generator design this value is the token's semantic attribute (often passed as `%union` / `YYSTYPE` in CUP).

---

e) Reserved words versus identifiers

For

```
cook cooking_time;
serve server;
```

the scanner produces:

```
COOK
IDENTIFIER
SEMICOLON
SERVE
IDENTIFIER
SEMICOLON
```

The two rules involved are:

```
"cook"                 { return Token.COOK; }
"serve"                { return Token.SERVE; }
[a-zA-Z][a-zA-Z0-9_]*  { return Token.IDENTIFIER; }
```

JFlex uses the **longest-match rule**: it chooses the rule that consumes the greatest number of input characters. If two rules match exactly the same number of characters, the rule that appears **first in the specification** wins.

- `cook` and the identifier pattern both match four characters. The reserved-word rule is earlier, so `cook` becomes `COOK`.

- `cooking_time` matches twelve characters as an identifier but only four characters as `cook`. The longest match is the identifier, so it becomes `IDENTIFIER`.
- `serve` and the identifier pattern both match five characters; the earlier `"serve"` rule gives `SERVE`.
- `server` matches six characters as an identifier and only five as `serve`; the longest match gives `IDENTIFIER`.

This is why reserved words do not need to be excluded from the identifier regex by hand: rule ordering resolves ties, and longest match handles longer identifiers that begin with a reserved word.

---

f) Scanner test — valid input

Input:

```
// Ugali preparation program

prepare ingredients;
boil 4 cups water;
add 2 cups flour;
stir;
add 1 cups flour;
stir;
press ugali;
turn ugali;
cook 8 minutes;
check consistency;
shape ugali;
serve ugali;
```

Token stream (`make test`, section `f`):

```
PREPARE
INGREDIENTS
SEMICOLON
BOIL
NUMBER(4)
CUPS
WATER
SEMICOLON
ADD
NUMBER(2)
CUPS
FLOUR
SEMICOLON
STIR
SEMICOLON
ADD
NUMBER(1)
CUPS
FLOUR
SEMICOLON
STIR
SEMICOLON
PRESS
UGALI
SEMICOLON
TURN
UGALI
SEMICOLON
COOK
NUMBER(8)
```

```
MINUTES
SEMICOLON
CHECK
CONSISTENCY
SEMICOLON
SHAPE
UGALI
SEMICOLON
SERVE
UGALI
SEMICOLON
```

The initial `//` comment and all whitespace produce no tokens.

---

g) Lexical error detection — invalid input

Input:

```
prepare ingredients;
boil 4 cups water;
add @2 cups flour;
cook 5# minutes;
```

`@` and `#` are not part of any valid token, so they are the two illegal characters. The scanner reports:

```
Lexical error at line 3, column 5: illegal character '@'
Lexical error at line 4, column 7: illegal character '#'
```

The complete token/error stream is:

```
PREPARE
INGREDIENTS
SEMICOLON
BOIL
NUMBER(4)
CUPS
WATER
SEMICOLON
ADD
Lexical error at line 3, column 5: illegal character '@'
NUMBER(2)
CUPS
FLOUR
SEMICOLON
COOK
NUMBER(5)
Lexical error at line 4, column 7: illegal character '#'
MINUTES
SEMICOLON
```

Line 3 is `add @2 cups flour;`: `add` occupies columns 1–3, the space is column 4, and `@` is column 5. Line 4 is `cook 5# minutes;`: `cook` occupies columns 1–4, the space is column 5, `5` is column 6, and `#` is column 7. The scanner recovers and continues after an illegal character, so the surrounding valid tokens are still produced.

---

h) Extension — multiline comments with lexical states

`Lexer.flex` declares an exclusive lexical state:

```
%x COMMENT
```

and uses it as follows:

```
"/*" {
  commentStartLine = line();
  commentStartColumn = column();
  yybegin(COMMENT);
}

<COMMENT> {
  "*/"      { yybegin(YYINITIAL); }
  [^*]+     { /* ignore comment body, including newlines */ }
  "*"       { /* ignore a lone '*' */ }
  <<EOF>>   { return Token.UNTERMINATED_COMMENT; }
}
```

The scanner starts in `YYINITIAL`. After the two-character sequence `/*` it switches to the exclusive `COMMENT` state using `yybegin(COMMENT)`. Normally the keywords, identifiers and other initial-state rules are not active in an exclusive state, so comment text can never be mistaken for a token. `*/` switches back with `yybegin(YYINITIAL)`. The `[^*]+` and `"*"` rules consume the body; a star followed by `/` is still matched by the longer `"*/"` rule, so the comment terminates correctly.

If the end of file is reached while still in `COMMENT`, the `<<EOF>>` rule returns `UNTERMINATED_COMMENT`. The scanner also remembers the line and column where the comment started, so the driver can print:

```
Lexical error at line 2, column 1: unterminated multiline comment
```

Example input:

```
/*
     Ugali should be stirred continuously.
     Add flour gradually.
*/
prepare ingredients;
boil 4 cups water;
```

produces only:

```
PREPARE
INGREDIENTS
SEMICOLON
BOIL
NUMBER(4)
CUPS
WATER
SEMICOLON
```

For the unterminated input

```
prepare ingredients;
/*
     This comment never ends.
```

the scanner produces `PREPARE INGREDIENTS SEMICOLON` and then the unterminated-comment error above. JFlex continues to update `yyline` and `yycolumn` as it consumes the comment, so error positions after a comment remain accurate.

---

i) Scanner and parser relationship (e.g. JFlex + CUP)

A JFlex scanner and a CUP parser normally communicate through a small, well-defined interface.

1. **Token generation.** The scanner's public method (`yylex()` here) returns one token at a time. Each token has an integer token code. In CUP these codes are usually the symbolic constants declared in the parser specification (`terminal BOIL, NUMBER, ...`) or are mapped from the scanner's token enum to those constants. CUP's generated parser calls `yylex()` whenever it needs the next lookahead symbol.
2. **Passing tokens to the parser.** The scanner object is given to the parser (for example, `parser.setScanner(scanner)`). The parser repeatedly calls the scanner and uses the returned code to select an action from its parse table. End of input is signalled by the scanner returning `null` (or a dedicated `EOF` token, depending on the interface).
3. **Token attributes.** A token code alone is not enough for `NUMBER(4)`: the parser also needs the value. The scanner exposes the semantic value of the current token, either through a `Lexer` field/method (`numberValue()`) or through a CUP `Symbol` whose `value` field is set before it is returned. In CUP the parser specification uses a `%union` and `%type` declarations to say which grammar symbols carry which attribute, so a production such as `Quantity ::= NUMBER CUPS` can read the integer from the `NUMBER` symbol. The parser then builds AST nodes such as `Quantity(4)`.
4. **Symbol table.** The scanner recognises identifiers, but the symbol table is usually managed by later phases. The parser or semantic-analysis phase inserts identifiers and attaches type/location information. The scanner can pass the identifier spelling (`yytext()` or an identifier attribute) to the parser; the parser/semantic phase then looks up or installs the symbol. Reserved words are not identifiers, so they are never interned as user names.
5. **Lexical errors versus syntax errors.** A lexical error means the input cannot be divided into valid tokens at all, e.g. `@` or an unterminated comment. The scanner should report it with a position and either skip the character and continue (as this scanner does) or abort. A syntax error means the tokens are valid but their arrangement violates the grammar, e.g. `boil four cups water;` (identifier where a number is required). The parser reports syntax errors, usually with the offending token and the expected token(s). Keeping these responsibilities separate makes both error messages more precise and lets the scanner recover from bad characters without confusing the parser.

A typical CUP/JFlex flow is therefore:

```
source text -> JFlex scanner -> token code + attribute -> CUP parser
                                                       -> AST/symbol table
```

The scanner only decides *what* the lexemes are; the parser decides whether their sequence is grammatical.

---

j) Analysis — `boil four cups water;`

Assume `four` matches `[a-zA-Z][a-zA-Z0-9_]*`.

1. **Tokens generated.** The scanner produces:

```
BOIL
IDENTIFIER(four)
CUPS
WATER
SEMICOLON
```

`four` is a valid identifier lexeme. It is not a reserved word and it is not an integer literal.

2. **Should the scanner report an error?** No. Each lexeme is legal according to the lexical specification. `four` is a well-formed identifier, so the scanner has no lexical reason to reject it. It would only report an error if the spelling could not match any token (for example `f0ur@` or an illegal character).
3. **Which phase detects the grammar violation?** The parser/syntax-analysis phase. The grammar requires

   ```
   Quantity -> NUMBER CUPS
   Boiling  -> BOIL Quantity WATER SEMICOLON
   ```

   so the parser expects `NUMBER` after `BOIL`, but it receives `IDENTIFIER`. This is a syntax error (a token-sequence error), not a lexical error. A semantic phase would only become relevant if the grammar allowed the construct and a later rule failed (for example a type or range check).
4. **Why this shows the separation of concerns.** The lexical analyser classifies characters into tokens and enforces only local, regular properties such as "identifiers start with a letter" and "integers contain digits". It does not know that a number is required at this point in a boiling instruction. Context-free structure — which token may follow which — belongs to the parser. The scanner correctly accepts `four` as an identifier; the parser correctly rejects the resulting token sequence because `IDENTIFIER` does not fit `Quantity -> NUMBER CUPS`. This division is exactly why lexical analysis and syntax analysis are separate compiler phases.

Verification summary

The original implementation reached part (g). It was checked as follows, with everything compiled and run:

- Parts (a)–(c): all reserved words, identifiers, integers, semicolons, whitespace and single-line comments are recognised by `Lexer.flex`.
- Part (d): `NUMBER(4)` is produced and `numberValue()` exposes the value.
- Part (e): `cook cooking_time` gives `COOK IDENTIFIER`; `serve server` gives `SERVE IDENTIFIER`, confirming longest match and rule priority.
- Part (f): the valid program produces exactly the token stream shown in section (f).
- Part (g): `@` and `#` are reported at line 3, column 5 and line 4, column 7.
- Part (h) is newly implemented with the `%x COMMENT` lexical state and `<<EOF>>` unterminated-comment detection.
- Parts (i) and (j) are answered above.

The original architecture (a JFlex `Lexer` returning a `Token` enum, driven by `Main.scala`) was preserved. Changes were additive: integer attributes, lexical states for multiline comments, a `Tests.scala` verifier, and the corrected identifier rule.